



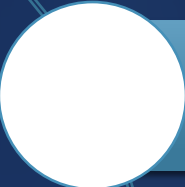
---

Tecnologías para el desarrollo de aplicaciones web

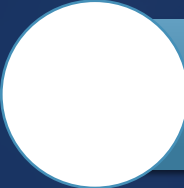
# CASCADING STYLE SHEETS

UNIDAD III

# CSS



Lenguaje que usamos para diseñar un documento HTML



Describe cómo deben mostrarse los elementos HTML



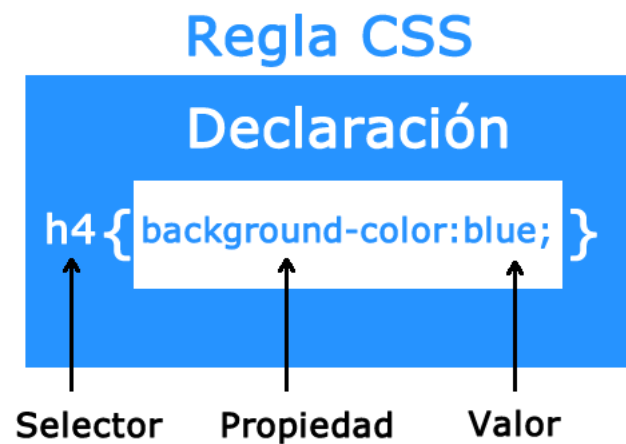
Facilita instrucciones para asignar estilos a los elementos HTML (colores, tipos de letra, tamaños)

# ESTILOS

- Los estilos se deben definir con CSS y luego se asignan a los elementos hasta que logremos el diseño visual que queremos para nuestra página.
- Un **estilo** se define declarando el **nombre de la propiedad** y su **valor** separados por dos puntos (:) y al final punto y coma (;).

`background-color:blue;`

Las **hojas de estilo** no son más que archivos de texto con la lista de reglas CSS que queremos asignar a los elementos del documento.



**Una regla es una lista de propiedades declaradas entre llaves e identificadas por un selector.**

VER ARCHIVO: [ESTRUCTURACSS.HTML](#)

## REGLA

LA MAYORÍA DE LAS PROPIEDADES CSS PUEDEN MODIFICAR UN SOLO ASPECTO DEL ELEMENTO (EL COLOR DE FONDO EN ESTE CASO). SI QUEREMOS CAMBIAR VARIOS ESTILOS AL MISMO TIEMPO, TENEMOS QUE DECLARAR MÚLTIPLES PROPIEDADES, ESTO LO LOGRAMOS CON UNA **REGLA**.





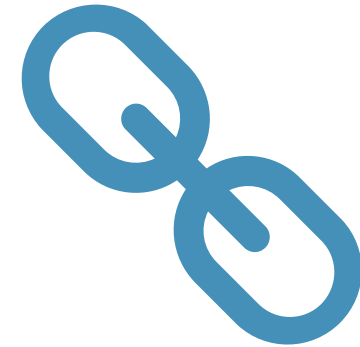
## Estilo en línea

Puede ser usado para aplicar un estilo único a un elemento en particular, éste se agrega en el **atributo style** y puede contener cualquier propiedad CSS



## Estilo interno

Se suele usar si una sola página HTML tiene un estilo único, éste es definido dentro del **elemento <style>**



## CSS externo

Cada HTML deberá incluir una referencia a la hoja de estilo externa dentro del **elemento <link>** o bien, importada con la **sentencia @import**

FORMAS DE INSERTAR ESTILOS O UNA HOJA DE ESTILOS (CSS)

# IMPORTACIÓN DE UNA HOJA DE ESTILO

- Una hoja de estilo puede ser *importada* con la sentencia **@import** de CSS. Esta sentencia puede usarse en un archivo CSS o dentro del elemento **style**:

```
<style type="text/css">  
  @import url(http://www.htmlhelp.com/estilo.css);  
  @import url(/hojasEstilo/estilo2.css);  
  ol{background-color: yellow; color: black;}  
</style>
```

Note que otras reglas de CSS pueden aún estar incluidas en el elemento **style**, pero todas las sentencias **@import** deben ocurrir al inicio de la hoja de estilo. Cualquier regla especificada en la misma hoja de estilo prevalecerá sobre reglas contradictorias en las hojas de estilo importadas. Por ejemplo, inclusive si una de las hojas de estilo importadas contuviera **ol{ background-color: brown}**, las listas ordenadas aún tendrían un fondo amarillo.

Utilizando el elemento **link** quedaría así:

```
<link rel="stylesheet" href="miestilo.css">
```

- El orden en que las hojas estilos son importadas es importante para determinar cómo trabajan en cascada. En el ejemplo de arriba, si la hoja de estilo importada **estilo.css** especifica que los elementos **strong** se muestren en rojo y la hoja de estilo **estilo2.css** especifica que los elementos **strong** se muestren en amarillo, la última regla prevalecería, y los elementos **strong** serían amarillos.

- **¿Qué estilo se usará cuando haya más de un estilo especificado para un elemento HTML?**

Todos los estilos se encadenarán en una nueva hoja de estilo virtual, con la siguiente prioridad:

1. En línea
2. Hojas de estilo internas y externas
3. Las predeterminadas del navegador

Ver archivo: [estructuraCSS2.html](#)

## PRIORIDAD DE LAS HOJAS DE ESTILO

# ORDEN EN CASCADA

- Una característica importante del CSS es que los **estilos se asignan en cascada**. Los elementos en los niveles bajos de la jerarquía heredan los estilos asignados a los elementos en los niveles altos. Por ejemplo:

```
section {  
    font-size:20px;  
}
```

**Regla en mi hoja de estilo**

```
<section>  
    <p>Mi texto</p>  
</section>
```

**Código en mi archivo HTML**

Por lo que, el texto del elemento **p** se mostrará con un tamaño de 20 píxeles, debido a que es un **elemento hijo** del elemento **section** y, por lo tanto, herede sus estilos.

## Comentarios en CSS

Son usados para explicar el código y pueden facilitarte la edición posteriormente. Son ignorados por los navegadores.

Comienzan con `/*` y terminan con `*/`



# SELECTORES SIMPLES

- **Seleccionan elementos basados en el nombre, id o clase.**

Una regla declarada con el nombre del elemento como selector afecta a todos los elementos de este tipo encontrados en el documento. Por ejemplo:

```
h2{font-style: italic}
```

Si queremos asignar los mismos estilos a elementos con nombres diferentes, es decir, diferentes selectores, los declaramos separados por una coma (,). Por ejemplo:

```
p,span{  
    font-size:15px;  
}
```

# SELECTORES SIMPLES

## Atributo id

Este atributo es un nombre, un identificador exclusivo del elemento y, por lo tanto, lo podemos usar para encontrar un elemento en particular dentro del documento. Para referenciarlo, **el selector debe incluir nombre del atributo precedido por el carácter #.**

```
#mitexto{  
    font-size:24px;  
}
```

## Atributo class

Este atributo puede asignarse a varios elementos dentro del mismo documento. Para referenciarlo, **el selector debe incluir el nombre del atributo precedido por un punto (.)**.

```
.mitexto{  
    font-size:24px;  
}
```

A un mismo elemento se le pueden asignar varias clases, para hacerlo solo hay que declarar los nombres de las clases separadas por un espacio: **class="negrita italica"**

- Existen otras combinaciones de selectores llamados ***selectores de descendencia o combinador*** porque afectan a elementos dentro de otros elementos (listando los selectores separados por un espacio), sin importar el lugar que ocupan en la jerarquía. *Ver archivo selectoresDescen.html y estilo2.css*

```
main p{  
    font-size:24px;  
}
```

- CSS incluye sintaxis adicionales para realizar una selección más precisa. Entre ellas podemos usar el carácter > **para referenciar un elemento que es hijo directo de otro elemento.**

```
section>p{  
    font-size:20px;  
}
```

La regla modifica a los elementos **p** que son hijos de un elemento **section**.

## SELECTORES DE DESCENDENCIA

- Con el carácter + **referenciamos un elemento que esta precedido por otro elemento**. Ambos deben de compartir el mismo elemento padre.

```
h1+p{  
    font-size:24px;  
}
```

La regla afecta a todos los elementos **p** que se ubican inmediatamente después de un elemento **h1**.

- Con el carácter ~ **creamos un selector que afecta a todos los elementos que se ubican a continuación de otro elemento**.

```
p~p{  
    font-size:20px;  
}
```

## SELECTORES COMBINADOR O DESCENDENCIA

| Selector                  | Example       | Example description                                                                         |
|---------------------------|---------------|---------------------------------------------------------------------------------------------|
| <u>.class</u>             | .intro        | Selects all elements with class="intro"                                                     |
| .class1.class2            | .name1.name2  | Selects all elements with both <i>name1</i> and <i>name2</i> set within its class attribute |
| .class1 .class2           | .name1 .name2 | Selects all elements with <i>name2</i> that is a descendant of an element with <i>name1</i> |
| <u>#id</u>                | #firstname    | Selects the element with id="firstname"                                                     |
| <u>*</u>                  | *             | Selects all elements                                                                        |
| <u>element</u>            | p             | Selects all <p> elements                                                                    |
| <u>element.class</u>      | p.intro       | Selects all <p> elements with class="intro"                                                 |
| <u>element,element</u>    | div, p        | Selects all <div> elements and all <p> elements                                             |
| <u>element element</u>    | div p         | Selects all <p> elements inside <div> elements                                              |
| <u>element&gt;element</u> | div > p       | Selects all <p> elements where the parent is a <div> element                                |
| <u>element+element</u>    | div + p       | Selects all <p> elements that are placed immediately after <div> elements                   |

Ver archivo: [selectoresDescen.html](#)

## SELECTORES COMBINADOR O DESCENDENCIA

# SELECTORES PSEUDO-CLASE Y PSEUDO-ELEMENTOS

## Pseudo-clase

Selecciona los elementos basados en un cierto estado.

### Sintaxis

```
selector:pseudo-clase{  
  propiedad:valor;  
}
```

Ver archivo: *selectoresPseudoClase.html*

## Pseudo-elemento

Es usado para diseñar partes específicas de un elemento.

### Sintaxis

```
selector::pseudo-elemento{  
  propiedad:valor;  
}
```

Ver archivo: *pseudoElementos.html*



# SELECTORES DE ATRIBUTO

## Selectores de atributo

Para situaciones en las que los atributos **id** y **class** no son suficientes para encontrar el elemento exacto que queremos modificar, **CSS** nos permite referenciar un elemento por medio de cualquier otro atributo que necesitemos. La sintaxis para definir esta clase de selectores incluye el nombre del elemento seguido del nombre del atributo entre corchetes:

### Sintaxis

```
selector[atributo]{  
    propiedad:valor;  
}
```

Por ejemplo:

```
ol[type]{  
    color:red;  
}
```

Esta regla modifica solo los elementos **ol** que tienen un atributo llamado **type**.

A éstos, también podemos agregarles el valor del atributo:

```
ol[type="A"]{  
    font-size:20px;  
}
```

CSS nos permite combinar el carácter = **con otros caracteres** para realizar una selección más específica. Los más utilizados son:

```
img[alt~="la"]{  
  transform:rotate(30deg);  
}
```

Referencia cualquier elemento **img** con un atributo **alt**, cuyo valor **incluye** la palabra la (p.e.: “la mascota”)

```
img[alt^="ma"]{  
  transform:rotate(60deg);  
}
```

Referencia cualquier elemento **img** con un atributo **alt**, cuyo valor **comienza** en “ma” (p.e.: “mascota”, “mascota”)

## SELECTORES ATRIBUTO

```
img[alt$="mi"]{  
  transform:rotate(30deg);  
}
```

Referencia cualquier elemento **img** con un atributo **alt**, cuyo valor **termina** en “mi” (p.e.: “sunami”)

```
img[title*="mi"]{  
  font-size:20px;  
}
```

Referencia cualquier elemento **img** con un atributo **title**, cuyo **valor contiene la cadena de caracteres** “mi” (p.e.: “textomicroche”)

Ver archivo: *selectoresAtributo.html*

## SELECTORES ATRIBUTO

## Colores CSS

Los colores son especificados **usando**:

- ✓ **Nombres de color predefinidos**
- ✓ Su **valor rgb** (rojo, verde, azul desde 0 hasta 255)
- ✓ Su valor **rgba** (rojo, verde, azul, alfa), el componente adicional es para definir la opacidad y el valor se puede declarar entre 0 (transparente) y 1 (opaco)
- ✓ Su **valor HEX** (hexadecimal)
- ✓ **HLS** (matiz, saturación, luminosidad)

*matiz* se expresa en grados de 0 a 360: entre 0 y 360 los rojos, cerca de 120 los verdes y cerca de 240 los azules.

*saturación* se representa en porcentaje, desde 0% (escala de grises) hasta 100% (todo color o totalmente saturado) y

*luminosidad* es un valor en porcentaje, desde 0% (completamente negro) a 100% (completamente blanco); un valor 50% representa una luminosidad promedio.

# COLORES

## Ejemplos:

**Color de fondo:** `style="background-color: DodgerBlue;"`

**Color de texto:** `style="color: Tomato;"`

**Color de borde:**

```
<h1 style="border: 2px solid Tomato;">Patty</h1>
```

Valor **RGB** y **HEX** de los colores:

```
rgb(255, 99, 71) style="background-color: rgb(255, 99, 71);"
```

```
#ff6347 style="background-color: #ff6347;"
```

Con **transparencia al 50%:**

```
rgba(255, 99, 71, 0.5)
```

Regla para obtener un color gris claro:

```
color: hsl(0,0%,80%);
```

Para encontrar los colores adecuados, pueden  
usar esta herramienta:

<https://htmlcolorcodes.com/>

# COLORES

# MODELO DE CAJA

Este término es usado en CSS cuando se habla de diseño y maquetación. El modelo de caja es una caja que envuelve a cada elemento HTML y está compuesto por:



Para obtener el **ancho** total de los elementos:

$$\text{AnT} = \text{ancho} + \text{padding izq} + \text{padding der} + \text{border izq} + \text{border der}$$

Para obtener el **alto** total de los elementos:

$$\text{ALT} = \text{alto} + \text{padding sup} + \text{padding inf} + \text{border sup} + \text{border inf}$$

Ver archivo: *boxModel.html*

# TAMAÑO

Por defecto, el tamaño de la mayoría de los elementos se determina según el espacio disponible en el contenedor. El ancho de un elemento se define como 100%, lo cual significa que será tan ancho como su contenedor y tendrá una altura predeterminada por su contenido.

CSS define las siguientes propiedades para declarar un tamaño personalizado:

**width** – declara el ancho de un elemento. El valor se puede especificar en píxeles, porcentaje o con la palabra clave auto (por defecto).

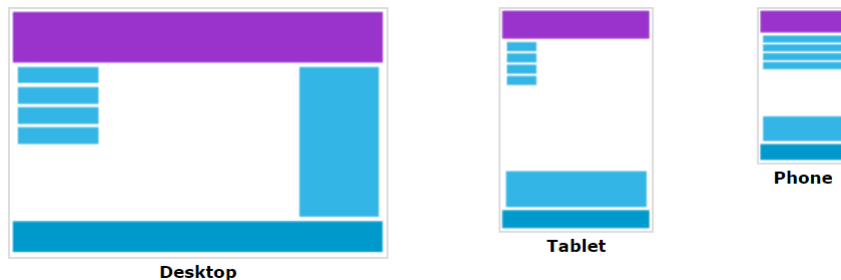
Cuando se especifica por porcentaje, el ancho se calcula según el navegador a partir del ancho del contenedor y cuando se declara con el valor auto, el elemento se expande hasta ocupar todo el espacio horizontal disponible dentro del contenedor.

**height** – declara la altura de un elemento. El valor se puede especificar en píxeles, porcentaje o con la palabra clave auto (por defecto). Cuando se especifica por porcentaje, el navegador calcula la altura del contenedor, y cuando se declara con el valor auto, el elemento adopta la altura de su contenedor.

## DISEÑO WEB ADAPTABLE (RESPONSIVE)

Este diseño nos permite que nuestra página web se vea bien en todos los dispositivos diferentes: computadoras de escritorio, tablets y teléfonos.

El diseño depende de los usuarios destino. El más común, es a una columna o una combinación como se muestra en la siguiente imagen:



Las páginas web no deben omitir información para adaptarse a dispositivos más pequeños, sino adaptar su contenido a cualquier dispositivo.

Se le llama diseño web adaptable cuando usa CSS y HTML para cambiar el tamaño, ocultar, reducir, ampliar o mover el contenido para que se vea bien en cualquier pantalla.



## ¿Qué es viewport (ventana gráfica)?

Es el área visible del usuario de una página web y varía según el dispositivo. Anteriormente, el diseño de las páginas web era estático y con un tamaño fijo, cuando comenzamos a navegar por internet usando tablets y teléfonos móviles, esas páginas fueron demasiado grandes para caber en la ventana gráfica.

La solución a ese problema fue que los navegadores de esos dispositivos redujeron toda la página web para adaptarse a la pantalla.

HTML5 introdujo un método para permitir que los diseñadores web tuvieran el control de la ventana gráfica, a través de la etiqueta `<meta>`, de la siguiente forma: **`<meta name = "viewport" content = "width = device-width, initial-scale = 1.0">`** y así, el navegador sabe cómo controlar las dimensiones y la escala de la página:

**width = device-width** establece el ancho de la página para seguir el ancho de la pantalla del dispositivo

**initial-scale = 1.0** establece el nivel del zoom inicial cuando el navegador carga la página por primera vez

# DISEÑO WEB ADAPTABLE (RESPONSIVE)

## ¿Qué es una Media Query?

Es una técnica CSS introducida en CSS3.

Hay que utilizar la regla **@media** para incluir un bloque de propiedades CSS solo si cierta condición es verdadera.

El resultado de la consulta es verdadero si el tipo de medio especificado coincide con el tipo de dispositivo en el que se muestra el documento y todas las expresiones en la consulta de medios son verdaderas. Cuando una consulta de medios es verdadera, se aplican la hoja de estilo o las reglas de estilo correspondientes, siguiendo las reglas normales en cascada.

A menos que use los operadores **only** o **not**, el tipo de medio es opcional y el tipo **all** estará implícito.

Los tipos de media son:

**all.-** Usado para todos los dispositivos de tipo multimedia

**print.-** Utilizado para impresoras

**screen.-** Se utiliza para pantallas de computadora, tabletas, teléfonos inteligentes, etc.

**speech.-** Se utiliza para lectores de pantalla que "leen" la página en voz alta

Ver archivo: *mediaQ.html*

# DISEÑO WEB ADAPTABLE (RESPONSIVE)

## Breakpoint

Podemos agregar un punto de ruptura (breakpoint) donde ciertas partes del diseño se comportarán de manera diferente en cada lado del punto de interrupción.

Para comenzar a construir un diseño grid responsive, hay que asegurar que todos los elementos HTML tengan la propiedad box-sizing establecida en border-box.

**\*{box-sizing: border-box;}**

Dicha propiedad nos permite decidir cómo se calculará el tamaño de un elemento y forzar a los navegadores a incluir el relleno y el borde en el tamaño declarado por la propiedad width.

Los valores disponibles son **content-box** y **border-box**.

Si un elemento flotado es más alto que el elemento que lo contiene, éste será desbordado fuera de su contenedor, para resolver este problema, agregaremos una clase con la propiedad de overflow.

**.clear{overflow: auto;}**

Ahora bien, sino podemos mantener el control de los márgenes y relleno (padding), se verán las barras de desplazamiento, por lo que, debemos usar el siguiente código:

# DISEÑO WEB ADAPTABLE (RESPONSIVE)

```
.clear::after{
```

```
    content: ""; /* se emplea para generar nuevo contenido de forma dinámica e  
    insertarlo en la página HTML, generalmente utilizada en los pseudoelementos :before y :after */
```

```
    clear: both; /* Restaura el flujo normal del documento y no permite que el  
    elemento siga flotando hacia la izquierda, la derecha o ambos lados (none, left, right y both)*/
```

```
    display: table;
```

```
}
```

*Ver archivo: [gridMedia.html](#)*

## Grid-View

Muchas páginas web están basadas en una grid-view, lo cual significa que la página está dividida en columnas y esto es muy útil al diseñar las páginas web, ya que facilita la colocación de elementos en la página.

Una grid-view comúnmente tiene 12 columnas y tiene un ancho total del 100% y se encogerá o expandirá a medida que cambie el tamaño del navegador.

Primero calculamos el porcentaje para cada columna  $100\% / 12 = 8.33\%$

Creamos una clase para cada una de las 12 columnas (class = "col-x") y un número que define cuántas columnas debe abarcar la sección.

Para ver su aplicación pueden consultar el ejercicio de la maqueta con media query trabajado en clase.

*Ver archivo: [grid-view.html](#)*

# DISEÑO WEB ADAPTABLE (RESPONSIVE)



# BOOTSTRAP

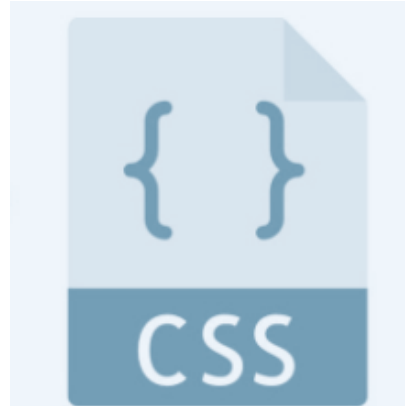
ES UN FRAMEWORK DE DESARROLLO  
FRONT-END DE CÓDIGO ABIERTO PARA LA  
CREACIÓN DE SITIOS Y APLICACIONES WEB.

VER SITIO: [HTTPS://GETBOOTSTRAP.COM/](https://getbootstrap.com/)



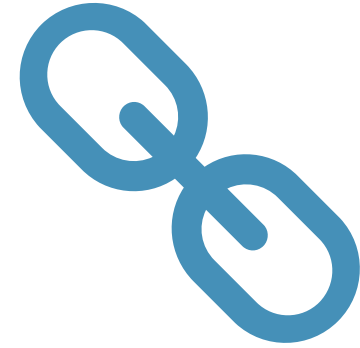
## INDEX

Debemos generar nuestro archivo `index.html` e incluir el elemento `<meta name="viewport">`.



## CSS

Incluir la hoja de estilo de Bootstrap, dentro de `<head>` para hacer uso de todo el diseño y la parte responsive.



## JS

Incluir JS en el elemento `<script>` antes de cerrar el elemento `<body>`.

Ver archivo: *index.html*

# UTILIZAR BOOTSTRAP



PROFESORA: PATRICIA  
ESCAMILLA MIRANDA